Q1. Create StudentDB database

```javascript
use StudentDB
db.createCollection("Students")
show dbs
```

Q2. Create Students, Faculty, and Courses collections

```javascript
db.createCollection("Faculty")
db.createCollection("Courses")
show collections
```

Q3. Insert a single student document

```javascript
db.Students.insertOne({
  RollNo: 101,
  Name: "Aarav Sharma",
  Department: "BCA",
  Semester: 5,
  CGPA: 8.4
})
```

Q4. Insert multiple student documents

```javascript
db.Students.insertMany([
```

```
  { RollNo: 102, Name: "Diya Verma", Department: "BCA", Semester: 5, CGPA: 8.1 },
  { RollNo: 103, Name: "Kabir Singh", Department: "BSc CS", Semester: 5, CGPA: 7.8 },
  { RollNo: 104, Name: "Meera Gupta", Department: "BCA", Semester: 5, CGPA: 9.0 },
  { RollNo: 105, Name: "Rohan Das", Department: "BCom", Semester: 5, CGPA: 7.5 }
])
```

Q5. Retrieve all student documents

```javascript
db.Students.find({})
```

Q6. Retrieve a student by Roll Number

```javascript
db.Students.findOne({ RollNo: 103 })
```

Q7. Retrieve students by Department

```javascript
db.Students.find({ Department: "BCA" })
```

Q8. Display only Name and Department

```javascript
db.Students.find(
  {},
  { _id: 0, Name: 1, Department: 1 }
)
```

Q9. Update the department of a specific student

```javascript
db.Students.updateOne(
  { RollNo: 103 },
  { $set: { Department: "BCA" } }
)
```

Q10. Modify the semester of students belonging to a department

```javascript
db.Students.updateMany(
  { Department: "BCA" },
  { $set: { Semester: 6 } }
)
```

Q11. Replace an existing student document

```javascript
db.Students.replaceOne(
  { RollNo: 105 },
  {
    RollNo: 105,
    Name: "Rohan Das",
    Department: "BCom",
    Semester: 5,
    CGPA: 7.7
  }
```

)

```

### Q12. Add an Email field to all student documents

```javascript

db.Students.updateMany(

 {},

 { $set: { Email: "student@example.com" } }

)

```

Q13. Remove an existing field from all documents

```javascript

db.Students.updateMany(

 {},

 { $unset: { Email: "" } }

)

```

Q14. Delete a specific student using Roll Number

```javascript

db.Students.deleteOne({ RollNo: 105 })

```

Q15. Delete multiple student records by condition

```javascript

db.Students.deleteMany({ Department: "BCom" })

```

Q16. Count the total number of student documents
```

```javascript
db.Students.countDocuments({})
```

Q17. Rename Students to StudentDetails

```javascript
db.Students.renameCollection("StudentDetails")
show collections
```

Q18. Create Departments collection and insert sample data

```javascript
db.createCollection("Departments")
db.Departments.insertMany([
  { DeptCode: "BCA", DeptName: "Computer Applications", HOD: "Dr. Sharma" },
  { DeptCode: "BSC", DeptName: "Computer Science", HOD: "Dr. Verma" },
  { DeptCode: "BCOM", DeptName: "Commerce", HOD: "Dr. Gupta" }
])
db.Departments.find({})
```

Q19. Drop the Departments collection

```javascript
db.Departments.drop()
show collections
```

Q20. Drop StudentDB database

Warning: This deletes the entire database and its data.

```javascript

db.dropDatabase()

show dbs

```

Q21. Recreate StudentDB and insert sample data

```javascript

use StudentDB

db.StudentDetails.insertMany([

  { RollNo: 201, Name: "Ananya Joshi", Department: "BCA", Semester: 5, CGPA: 8.6 },

  { RollNo: 202, Name: "Vivaan Mehta", Department: "BSc CS", Semester: 5, CGPA: 8.2 }

])

db.StudentDetails.find({})

```

Q22. Perform CRUD operations using MongoDB Compass

```text

CREATE:

Open StudentDetails → Add Data → Insert Document.

READ:

Use the Filter field:

{ "Department": "BCA" }

UPDATE:

Open a document → Edit → Change a field → Update.

DELETE:

Select a test document → Delete → Confirm.

VERIFY IN MONGOSH:
```

```
db.StudentDetails.find({})

db.StudentDetails.countDocuments({})
```

Q23. Demonstrate database and collection management

```javascript
use StudentDB

show collections

db.createCollection("DemoCollection")

show collections

db.DemoCollection.drop()

show collections
```

Q24. Compare MongoDB CRUD operations with SQL

```text
1. INSERT

MongoDB:

db.Students.insertOne({RollNo: 1, Name: "Asha"})

SQL:

INSERT INTO Students (RollNo, Name)

VALUES (1, 'Asha');

2. READ

MongoDB:

db.Students.find({RollNo: 1})

SQL:

SELECT * FROM Students WHERE RollNo = 1;
```

3. UPDATE

MongoDB:

db.Students.updateOne(

 {RollNo: 1},

 {$set: {Name: "Asha K."}}

)

SQL:

UPDATE Students

SET Name = 'Asha K.'

WHERE RollNo = 1;

4. DELETE

MongoDB:

db.Students.deleteOne({RollNo: 1})

SQL:

DELETE FROM Students WHERE RollNo = 1;

DIFFERENCES:

MongoDB stores data in documents and collections.

SQL databases generally store data in rows and tables.

MongoDB uses query objects and operators.

SQL uses structured query statements.

Both support CRUD operations.

```

### Q25. Comprehensive laboratory report

```text

TITLE:

NoSQL Database Design Using MongoDB

OBJECTIVE:

To understand database creation, collection management,

CRUD operations, and database management using MongoDB.

REQUIREMENTS:

1. MongoDB Community Server

2. MongoDB Shell (mongosh)

3. MongoDB Compass

4. Ubuntu/Linux operating system

5. Basic database knowledge

PROCEDURE:

1. Created StudentDB database.

2. Created Students, Faculty, and Courses collections.

3. Inserted single and multiple student documents.

4. Retrieved all documents and specific student records.

5. Used projection to display selected fields.

6. Updated and replaced student documents.

7. Added and removed document fields.

8. Deleted individual and multiple records.

9. Counted documents and renamed a collection.

10. Created and dropped collections.

11. Dropped and recreated the database.

12. Performed CRUD operations using MongoDB Compass.

13. Compared MongoDB operations with SQL.

OBSERVATIONS:

insertOne() inserts one document.

insertMany() inserts multiple documents.

find() retrieves documents.

findOne() retrieves a single document.

updateOne() updates one document.

updateMany() updates multiple documents.

replaceOne() replaces a document.

$set adds or updates fields.

$unset removes fields.

deleteOne() deletes one document.

deleteMany() deletes multiple documents.

countDocuments() counts documents.

drop() deletes a collection.

dropDatabase() deletes a database.

CONCLUSION:

The experiment demonstrated MongoDB database creation,

collection management, and CRUD operations. It also

explained how MongoDB Shell and Compass can be used

to manage data and how MongoDB differs from SQL databases.

SCREENSHOTS:

Attach screenshots of database creation, document

insertion,

retrieval, updates, deletion, MongoDB Compass operations,

and database recreation.